NPU-Native Neural Networks on AMD XDNA 2

Hugo Penedones

Draft — April 21, 2026

High-throughput residual inference for HIGGS on AMD XDNA 2

Abstract

We study a deliberately narrow question: which neural-network shape best matches the AMD XDNA 2 NPU for high-throughput, low-latency inference on dense collider-style features? We answer it with a forward-only residual MLP whose hidden body streams across the 32 compute tiles of the Strix Point array. On the public HIGGS benchmark, we retain three key configurations: a host-assisted $H = 32$, $L = 8$ model that sustains about 2.38M samples/s wall-clock, a retrained fully on-array $H = 32$, $L = 30$ model that reaches 76.61% test accuracy and 14.08M samples/s wall-clock at $B = 64$ (2.44M at the native streaming batch $B = 8$), and a full-data $H = 64$, $L = 32$ model trained on 10.4M examples that reaches 77.98% test accuracy, 0.865 ROC AUC, and 0.877 PR AUC on the fixed 500k-example test split. The design is small enough to understand end to end yet strong enough to support a credible systems case for dense event filtering on consumer hardware. The contribution is a measured co-designed operating regime, not a production trigger deployment, a state-of-the-art HIGGS accuracy paper, or an energy-efficiency study.

1 Introduction

Modern NPUs become most interesting when they stop being treated as "small GPUs" and start being programmed as spatial dataflow machines. Instead of launching one large shared-memory kernel, the programmer assigns a small local job to one tile, hands the activations to the next tile, and repeats until the full network has been traversed. On AMD XDNA 2, the programmer sees a 2D array of compute tiles, each with local SRAM, a VLIW plus SIMD core, a matrix-multiply unit, and explicit routing. The central question is therefore no longer just "what architecture is accurate?", but also "what architecture maps cleanly to many tiny memories connected by explicit streams?".

The focus here is deliberately narrow: a residual MLP for HIGGS, with one residual matrix per tile and a forward-only spatial-streaming deployment on the NPU. The aim is not to survey every model family or every use of XDNA 2, but to isolate one design that can be understood end to end.

HIGGS is a natural benchmark for this study. It is a dense 28-feature binary classification task derived from collider events, introduced by Baldi, Sadowski, and Whiteson as a large-scale signal-versus-background benchmark with roughly 11 million examples [6, 7]. That makes a residual MLP structurally plausible before any hardware argument is made. It also makes the throughput story easier to defend. For example, the CMS Phase-2 Level-1 trigger must inspect detector data at the 40 MHz bunch-crossing rate, produce decisions with a latency budget of about 12.5 μs, and emit accepted events at roughly 750 kHz [9]. Those rates explain why multi-megasample-per-second measurements are interesting here, but they are not used to imply deployment equivalence: the best wall-clock repeated-sample result in this paper is 14.08M samples/s at $B = 64$ on a fully on-array $H = 32$, $L = 30$ path, while the same model reaches 2.44M samples/s at the native $B = 8$ streaming batch. Those figures sit above the

accepted-event rate yet far below the full bunch-crossing stream. The point is to study the same general systems regime of dense features, strict latency sensitivity, and pressure for very high sustained throughput.

This places the paper next to three adjacent literatures. For HIGGS itself, the reference landscape already includes both the original deep-versus-classic baseline framing and the strong tabular baselines now typically led by boosted tree ensembles [6, 13, 14, 15, 16, 17]; attention-based and decision-tree-hybrid deep tabular methods such as TabNet and NODE have also been evaluated on this benchmark [21, 22]. Comprehensive surveys of machine learning for particle physics provide broader context for where such dense-event classification sits within the HEP community [23]. For low-latency collider inference, substantial prior work has targeted FPGA-based trigger deployments and related co-design flows; the hls4ml open codesign toolchain makes the path from a trained model to firmware particularly direct [10, 19, 9]. At the accelerator level, the closest precedent is the spatial/dataflow tradition, where local SRAM and explicit movement constrain the network shape as much as raw arithmetic throughput does; purpose-built DNN accelerators and the Google TPU family both illustrate how data movement can become the primary design constraint rather than arithmetic throughput [11, 12, 20]. The claim here is narrower: given an existing benchmark, existing consumer NPU hardware, and an existing compiler/runtime stack, what is the simplest residual design that maps cleanly enough to the array to make a credible end-to-end systems case?

2 The hardware model we program against

Figure 1 shows the physical layout targeted here. The Ryzen AI 9 HX 370 exposes 32 compute tiles arranged as 8 columns by 4 compute rows. Below them sit 8 MemTiles, and below those 8 shim tiles that connect the array to host-visible memory and runtime control.

Property	Why it matters here
32 compute tiles (8 columns × 4 rows)	Depth maps naturally to tile count; an $L = 8$ body uses 2 columns, an $L = 32$ body uses the full array.
About 64 KB local SRAM per compute tile [4, 3]	Parameters must stay small enough to live locally; that strongly constrains hidden width.
8×8 MMUL-friendly blocked execution	Widths and batch sizes should be divisible by 8, and host-side packing must respect tiled layouts.
Explicit FIFOs, DMA, and routing	Data movement is part of the design, not an afterthought hidden behind a cache hierarchy.
IRON / MLIR-AIE / XRT software stack	The implementation programs the array close to the hardware model rather than through a high-level opaque runtime.

Table 1: Only a few hardware facts matter for this paper. The most important is not the marketing TOPS number, but the combination of tiny tile memories and explicit data movement.

These constraints shape the model directly. The hidden body is kept square so that each tile stores one $H \times H$ residual weight matrix. Widths are kept as multiples of 8 to match the AIE MMUL datapath. The host only moves the data that are still cheaper to keep on CPU today: the input projection and, for the fastest retained variant, the tiny final $H \to 2$ classifier head.

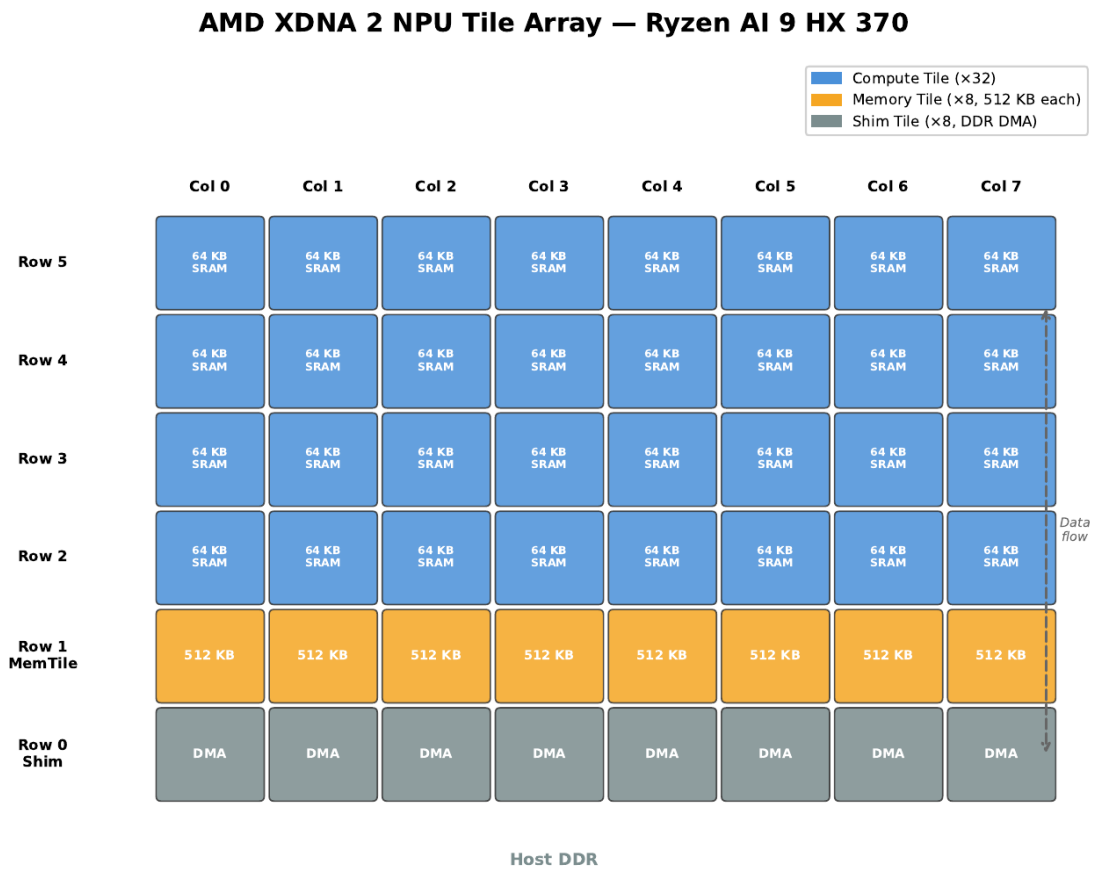

Figure 1: Physical layout of the AMD XDNA 2 NPU used here. The retained design can use all 32 compute tiles; the fastest variant uses only 2 columns, while the deeper $L = 32$ body occupies the full 8-column array.

3 A residual network that matches the array

The network studied here is intentionally minimal.

Notation

We use H for hidden width, L for the number of residual blocks, and B for batch size. In the deployment path studied here, one residual block maps to one compute tile. We use $\to$ for layer mappings (for example $H \to 2$ classifier) and $\times$ for matrix shapes (for example $H \times H$ weights).

Model definition

$$h_{\ell+1} = \mathrm{ReLU}(h_\ell W_\ell) + h_\ell, \qquad \ell = 0, \ldots, L-1.$$

The skip connection follows the convention introduced by He et al. [18]; each residual matrix W_ℓ is assigned to one compute tile. Activations enter the array, flow through the tiles in a snake, and return to the host only after the full hidden body has completed. In the current deployment path:

- the host CPU runs the input projection from the native 28 HIGGS features to hidden width H,
- the NPU runs the residual body,
- the host CPU currently keeps the tiny final $H \to 2$ head when that is faster than the available fused on-array tail.

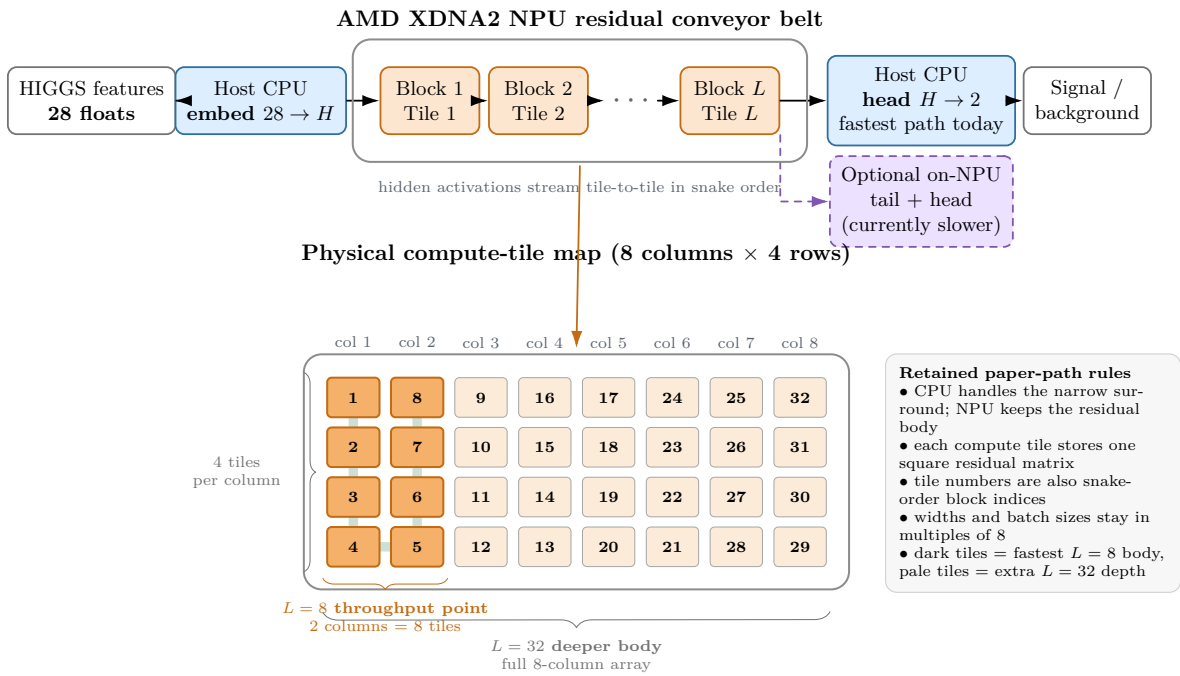

Figure 2: Model-to-hardware mapping for the retained HIGGS/XDNA 2 deployment path. The host CPU keeps the cheap $28 \to H$ input projection and, in the fastest retained path, the tiny $H \to 2$ head. The NPU assigns one $H \times H$ residual block per compute tile, using 2 columns for the $H = 32$, $L = 8$ speed-optimized variant and the full 8-column array for the deeper $L = 32$ body.

This design leads to two retained operating variants.

- **Speed-optimized variant:** $H = 32$, $L = 8$, mapped to 2 columns. This is the fastest end-to-end HIGGS point measured so far.
- **Accuracy-tuned variant:** $H = 64$, $L = 32$, trained on the full public dataset and mapped to the full 8-column array when the deeper body is streamed on the NPU.

The retained paper path keeps the narrow surround on CPU and uses the NPU only for the residual streaming body. The same snake-order placement then scales from the 2-column $L = 8$ speed-optimized variant to the full 8-column $L = 32$ body; Figure 2 makes that deployment contract concrete.

The important design choice is not just "use an MLP", but "use an MLP whose depth and width map directly to the array's tile structure". That keeps the design simple enough to analyze and direct enough to deploy.

4 From Python model to NPU artifact

The toolchain is deliberately short. `aie_kernels/matmul_relu_skip.cc` defines the tile-local residual kernel. `resmlp/streaming_design.py` turns a chosen (H, B, L) shape into a spatial-streaming MLIR-AIE program. `resmlp/streaming_op.py` wraps the compiled XCLBIN and instruction binary as a resident XRT operator. Finally, `resmlp/streaming_infer.py` drives trained checkpoints through that operator and measures wall-clock throughput.

The novelty claimed here is not a new compiler or runtime. IRON and MLIR-AIE already provide the compilation substrate [1, 2]; what is new is the model-to-array mapping, the host/NPU partition, and the end-to-end measurements of that co-designed path on real HIGGS checkpoints.

A key detail of the current forward path is that the residual weights are embedded at compile time. Each throughput run therefore builds a checkpoint-specific streaming artifact. That is not the most general deployment contract. It is, however, the cleanest way to study the geometry,

routing, and wall-clock behavior of a resident spatial-streaming design.

5 Training protocol on HIGGS

The data pipeline uses the public `jxie/higgs` mirror of the published train/test splits [8]. The local cache materializer produces a fixed 10.5M-example train split and a fixed 500k-example test split. For model selection, training holds out 100k examples from the train split. That leaves 10.4M examples for optimization. All reported runs use the native 28-dimensional feature vector without any padded-width preprocessing. Hyperparameter search and checkpoint selection use held-out validation accuracy. The 500k-example test split is used only for final evaluation of already-selected models.

HIGGS is also a good benchmark because it sits at the intersection of two real literatures. In high-energy physics it is a canonical dense-event benchmark [6], situated within a broader programme of applying machine learning to particle-physics classification and reconstruction that is surveyed in [23]. In tabular machine learning it is one of the standard large-data problems on which tree ensembles such as XGBoost, LightGBM, and CatBoost remain strong baselines, while modern deep tabular models—including attention-based and decision-tree-hybrid approaches—are now more competitive but not universally dominant [13, 14, 15, 16, 17, 21, 22]. That means the relevant bar is not "beat every HIGGS leaderboard", but "land in a statistically credible part of the baseline landscape while preserving a deployment shape the NPU can actually exploit". Good-enough accuracy on HIGGS is therefore genuinely worth caring about even if the main contribution of this work remains systems-oriented.

Model	Training recipe
$H = 32$, $L = 8$ full-data pilot	5 epochs on the AMD Radeon 890M ROCm path; AdamW, batch 8192, cosine $3 \times 10^{-3} \rightarrow 10^{-4}$, weight decay 10^{-4}, no label smoothing.
$H = 32$, $L = 32$ long schedule	20 epochs on the AMD Radeon 890M ROCm path; AdamW, batch 8192, cosine $3 \times 10^{-3} \rightarrow 10^{-4}$, weight decay 10^{-4}, no label smoothing.
$H = 64$, $L = 32$ validation-selected tuning run	Best checkpoint at epoch 46 of a 50-epoch MLflow/Optuna sweep on the AMD Radeon 890M ROCm path; AdamW, batch 4096, cosine $1.069 \times 10^{-3} \rightarrow 8.70 \times 10^{-5}$, weight decay 3.23×10^{-3}, label smoothing 0.0197.

Table 2: Training recipes for the reported HIGGS checkpoints. Both direct runs and validation-driven hyperparameter search were used to produce the reported models.

6 Results

6.1 Accuracy on the full public HIGGS split

Table 3 summarizes the current full-data results. The key point is that the systems design is not accuracy-indifferent. Even this deliberately hardware-friendly residual MLP becomes a credible classifier once it is trained on the full public HIGGS data.

Two key findings emerge. First, depth helps. Moving from $L = 8$ to $L = 32$ improves every metric even before any serious hyperparameter search. Second, validation-driven hyperparameter tuning lifts the result from the high-0.85 ROC band to 0.865 ROC AUC while also improving accuracy, PR AUC, and log loss. This is still not a claim of state-of-the-art HIGGS accuracy, but it is strong enough that the main contribution of the paper is no longer accuracy-indifferent.

6.2 Throughput of the spatial-streaming deployment

Table 4 reports repeated-sample throughput over 50M processed samples on the retained HIGGS checkpoints. The "sync" and "async" wall-throughput columns are end to end, including the host input projection, host head, all host-visible runtime overheads, and the NPU residual stage. The "kernel" column is measured separately by the benchmarking CLI with an isolated dispatch+wait calibration on the same compiled bitstream, so the final ratio column quantifies how much of that pure kernel ceiling survives in the overlapped async path.

6.2.1 Reducing host-loop overhead with a double-buffered host loop

Earlier revisions of this paper reported that wall throughput was several times lower than kernel throughput for every configuration. Instrumenting the host loop showed that the gap was dominated by Python per-call overhead: the $H = 64$, $L = 32$ configuration spent roughly 380 µs/call on the CPU input projection, 64 µs/call tiling the activations, 78 µs/call detiling outputs, and 71 µs/call on the CPU head, against only 156 µs/call in kernel wait. Two changes attack this gap without changing the NPU kernel or the network itself. First, we replaced 32 small Python dispatches per kernel invocation with four stacked numpy operations (one embed matmul on all $S \times B$ rows, one reshape/transpose pack, one reshape/transpose detile, one head matmul), which collapses the host work from ∼590 µs/call to roughly 116 µs/call on the $H = 64$, $L = 32$ checkpoint. Second, we use two-slot ping-pong device buffers and dispatch the next kernel with `run = xrt_kernel(...)` before the host begins consuming the previous slot's output, so a large fraction of the host work overlaps with kernel execution instead of paying the full host + kernel sum on every iteration.

The corrected measurements now show four different regimes. For the two hybrid $H = 32$ variants the async host loop materially improves wall throughput over the serial baseline and

Model	Test acc.	ROC AUC	PR AUC	Log loss
$H = 32$, $L = 8$ full-data (5 epochs)	75.89%	0.8430	0.8559	0.4861
$H = 32$, $L = 32$ full-data (5 epochs)	76.56%	0.8502	0.8628	0.4762
$H = 32$, $L = 32$ full-data (20 epochs)	76.98%	0.8542	0.8668	0.4705
$H = 32$, $L = 30$ full-NPU (epoch 9)	76.61%	0.8507	0.8635	0.4759
Validation-selected tuning run: $H = 64$, $L = 32$	77.98%	0.8653	0.8770	0.4543

Table 3: Full-data HIGGS results on the public `jxie/higgs` mirror. Validation-selected tuning at $H = 64$, $L = 32$ reaches 0.865 ROC AUC while remaining within the same residual design family.

Configuration	Wall throughput		Kernel throughput	Wall/kernel
	sync	async	isolated	(async)
$H = 32$, $L = 8$, 2 columns	1.72M	2.38M	2.44M	97%
$H = 32$, $L = 32$, 8 columns	1.64M	2.25M	2.27M	99%
$H = 64$, $L = 32$, 8 columns	0.82M	1.45M	1.99M	73%

Table 4: Repeated-sample throughput on the retained HIGGS checkpoints. "Sync" and "async" refer only to the host-loop implementation; all columns use the same compiled NPU bitstream. "Kernel throughput" is measured with a separate isolated dispatch+wait calibration rather than inferred from the overlapped async loop.

nearly closes the gap to isolated kernel throughput: the $L = 8$ speed variant rises from 1.72M to 2.38M samples/s and lands at 97% of isolated kernel throughput, while the deeper $H = 32$, $L = 32$ variant rises from 1.64M to 2.25M and lands at 99%. The retrained full-NPU $H = 32$, $L = 30$ path goes one step further by eliminating the CPU embed/head matmuls entirely. At $B = 8$ it reaches 2.44M samples/s wall-clock and 99% of isolated kernel throughput, a modest but real gain over the host-assisted $H = 32$, $L = 32$ baseline. At $B = 64$, the same fully on-array graph scales to 14.08M samples/s wall-clock while still landing at 99.6% of isolated kernel throughput. The wider $H = 64$, $L = 32$ accuracy variant also improves substantially—from 0.82M to 1.45M samples/s (+78%)—but still reaches only 73% of isolated kernel throughput. There the host path remains a real wall-time cost: the benchmark reports about 72 µs/call in pre-dispatch host work and 43 µs/call in post-wait host work against a 128 µs/call isolated kernel.

This sharpens the remaining "NPU-native" question rather than eliminating it. For narrow $H = 32$ bodies, moving the input projection and tiny head onto the array is now shown to help in two distinct ways: it removes most of the host-visible work at $B = 8$ and opens a much steeper scaling curve at larger batches, where the fully on-array path reaches 14.08M samples/s. For the wider $H = 64$ configuration, however, host-side work still consumes a substantial fraction of the wall path, so further host/runtime optimization or a cleaner on-array ingress/egress path for that wider regime could still buy real throughput.

Figure 3 combines the retained full-data accuracy rows with the measured async wall-throughput rows. It makes the trade-off explicit: the retrained full-NPU $H = 32$, $L = 30$ path is now the best-throughput point at the native $B = 8$ streaming batch, $H = 32$, $L = 32$ trades some speed for slightly better accuracy, and the tuned $H = 64$, $L = 32$ run remains the accuracy ceiling, but at 1.45M samples/s wall-clock it sits well below the two best $H = 32$ variants.

6.3 CPU throughput baseline

To put the NPU throughput numbers in context, we measure the same models on a laptop CPU using both PyTorch and a decomposed NumPy path that mirrors the NPU's inference structure (separate embed, sequential residual body, head).

Hardware and software. The baseline machine is the same AMD Ryzen AI 9 HX 370 (12 cores / 24 threads, max 5.16 GHz) used for all NPU measurements, running Linux 6.17. PyTorch

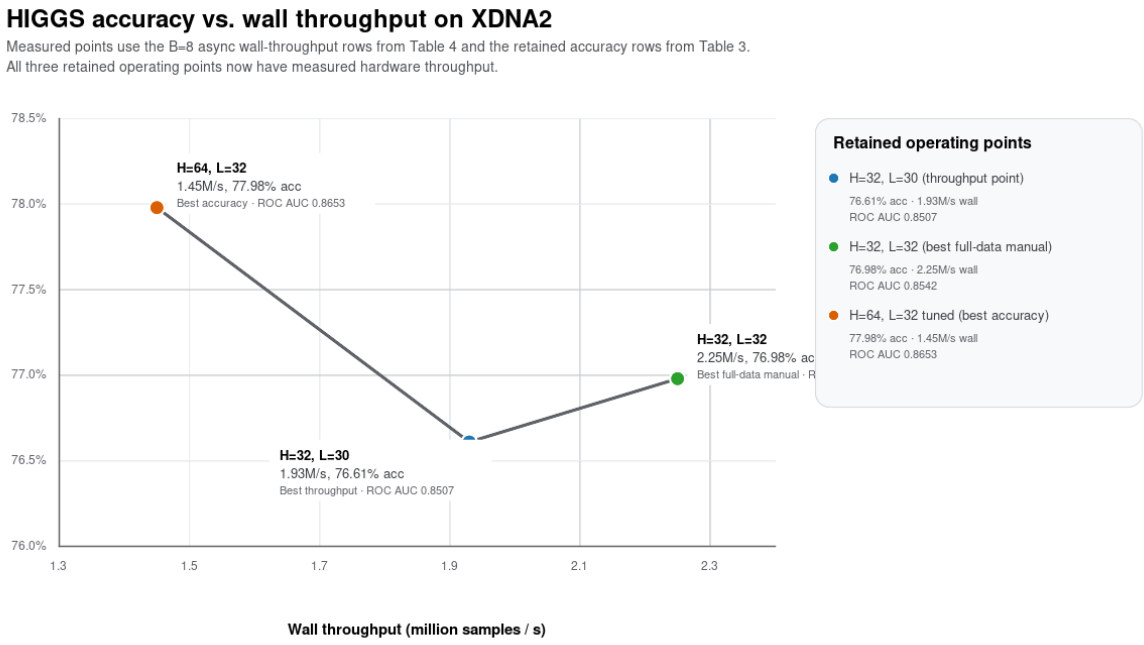

Figure 3: Accuracy-throughput frontier for the retained HIGGS/XDNA 2 variants. X-axis: test accuracy; Y-axis: wall-clock throughput in M samples/s. Throughput is the async/double-buffered measurement from Table 4.

Configuration	Batch size	PyTorch CPU	NumPy decomposed
$H = 32$, $L = 8$ (speed)	$B = 8$	104k samples/s	491k samples/s
$H = 32$, $L = 8$ (speed)	$B = 256$	1.82M samples/s	3.41M samples/s
$H = 32$, $L = 8$ (speed)	$B = 1024$	3.27M samples/s	1.73M samples/s
$H = 64$, $L = 32$ (accuracy)	$B = 8$	28k samples/s	105k samples/s
$H = 64$, $L = 32$ (accuracy)	$B = 256$	352k samples/s	197k samples/s
$H = 64$, $L = 32$ (accuracy)	$B = 1024$	598k samples/s	234k samples/s

Table 5: CPU throughput baseline for the same model architectures, measured on the AMD Ryzen AI 9 HX 370 host (12 cores) used for all NPU experiments. PyTorch uses 12 intra-op threads; NumPy uses OpenBLAS with default threading. All runs process 5M samples with random `float32` inputs.

2.10.0 uses 12 intra-op threads (the physical core count); NumPy 1.26.4 is backed by OpenBLAS with the same default thread pool. No explicit thread pinning or frequency governor tuning was applied; the measurements reflect the default out-of-the-box configuration.

Methodology. Each benchmark processes 5M samples using random `float32` inputs of the correct dimensionality ($D = 28$). Ten warmup calls precede the timed region. The PyTorch path calls `model.forward()` under `torch.no_grad()`; the NumPy path applies the embed, residual chain, and head as explicit `numpy.matmul` calls, matching the host/NPU split used in the streaming deployment. We sweep batch sizes from $B = 8$ (matching the NPU's streaming batch) through $B = 1024$ to show how CPU throughput scales with batch-level parallelism.

Several patterns are worth noting. First, the CPU strongly favors large batch sizes: PyTorch throughput on the speed variant improves by $\sim 31\times$ going from $B = 8$ to $B = 1024$, because larger batches amortize per-call dispatch overhead and expose enough work for BLAS multi-threading. At $B = 8$, the 32×32 matmuls are too small for OpenBLAS to distribute across cores, and PyTorch's per-layer dispatch adds measurable overhead; at larger batch sizes, OpenBLAS saturates the 12 available threads and PyTorch's fused ATen kernels pay off. Second, the NumPy

Configuration	Batch size	Sync wall	Async wall	Kernel
$H = 32$, $L = 30$ full-NPU	$B = 8$	1.93M	2.44M	2.48M
$H = 32$, $L = 30$ full-NPU	$B = 64$	8.34M	14.08M	14.16M

Table 6: Batch-scaling of the retrained fully on-array $H = 32$, $L = 30$ path. At the native streaming batch ($B = 8$), moving embed/head on-array yields a modest wall-throughput gain over the hybrid $H = 32$, $L = 32$ path; at larger batches the same graph scales to 14.08M samples/s while remaining very close to its isolated kernel throughput.

decomposed path is faster than PyTorch at small batch sizes (lower per-call overhead) but slower at large batch sizes where PyTorch's internal parallelism dominates.

Third, and most relevant to the NPU comparison: at the NPU's native streaming batch size of $B = 8$, the CPU reaches only 104k samples/s (PyTorch) for the speed variant—roughly 23× slower than the hybrid NPU path's 2.38M samples/s wall-clock throughput and 23.5× slower than the retrained fully on-array path's 2.44M samples/s. Even at the CPU's best batch size ($B = 1024$), the peak of 3.27M samples/s is still far below the full-NPU $B = 64$ result of 14.08M samples/s. For the accuracy variant ($H = 64$, $L = 32$), the NPU retains a clear advantage: the CPU peaks at 598k samples/s versus the NPU's 1.45M samples/s wall-clock, a 2.4× factor.

Note. Both the CPU baseline and the NPU throughput measurements were obtained on the same AMD Ryzen AI 9 HX 370 host, making the comparison controlled: the two results share the same memory subsystem, OS, and software stack, differing only in whether the residual body runs on the host cores or the on-die NPU array.

7 Discussion

The claim is deliberately narrow but defensible.

- A residual MLP can be shaped to match the AMD XDNA 2 array very directly: one residual matrix per tile, streamed activations, explicit routing, and checkpoint-specific resident weights.
- That design already delivers multi-million-sample-per-second inference on a real collider-style dense benchmark using cheap consumer hardware, culminating in 14.08M samples/s on a retrained fully on-array $H = 32$, $L = 30$ path.
- The same design family also reaches nontrivial full-data HIGGS quality, culminating in 77.98% test accuracy and 0.865 ROC AUC on the fixed 500k-example test set.

Just as importantly, the paper avoids claims it does not need. It does not present the current model as a production trigger deployment, the current HIGGS result as state of the art, or the present measurements as an energy-efficiency study. The benchmark is public, the hardware is existing AMD XDNA 2, and the compiler/runtime substrate is pre-existing; none of those ingredients are introduced here. The novelty is the co-designed operating regime: a residual MLP whose hidden-body geometry matches the array closely enough that explicit spatial deployment yields both a useful throughput result and a statistically credible HIGGS checkpoint.

A useful boundary from the SRAM accounting is that wider hidden bodies and wider surrounding inputs do not stress the current deployment in the same way. For the present $B = 8$ spatial-streaming layout, an internal residual tile carries roughly $1024 + 2H^2 + 32H$ bytes, while the ingress tile carries $1024 + 2H^2 + 64H$ because it must hold both the host-ingress buffers and the first residual stage. That makes the first tile, not the middle of the belt, the width bottleneck. The retained layout still fits at $H = 160$ with about 62.5 KB live on the ingress tile, but the same accounting already pushes $H = 168$ past 64 KB there even though the internal tiles would

still fit. In the current paper, then, widening the hidden body is a genuine tile-SRAM limit of the current spatial-streaming placement.

The surrounding stages are different. For the native 28-feature HIGGS input, even a padded $32 \to H$ input projection carries only $64H$ bytes of bf16 weights (4 KB at $H = 64$, 10 KB at $H = 160$), so keeping that stage on CPU today should not be read as evidence of an SRAM bottleneck. The awkward part is that 28 inputs do not naturally match the 8×8 MMUL granularity, so an on-array ingress path must pay packing, alignment, and boundary-management costs. The same distinction is even clearer for the tiny $H \to 2$ head: its weight footprint is negligible, and the measured slowdown of the fused NPU head is best understood as synchronization and launch overhead. This keeps the scope narrow. Within the HIGGS-style feature regime studied here, wider surrounding inputs mainly stress alignment and runtime boundaries, whereas the hard SRAM wall comes from widening the hidden body under the present first-tile layout.

8 Conclusion and next steps

The main result is simple: a network designed for spatial compute on AMD XDNA 2 can deliver unusually high throughput on real dense-event classification while remaining small enough to understand and validate end to end. The model family is deliberately modest, but it maps cleanly to the array and now yields three concrete operating points: a host-assisted speed path, a fully on-array $H = 32$, $L = 30$ path that reaches 14.08M samples/s, and a nontrivial full-data $H = 64$, $L = 32$ HIGGS result.

What exists today is a measured prototype family spanning both host-assisted and fully on-array inference paths; energy-efficiency characterization remains future work.

The next steps are correspondingly focused. The double-buffered host loop (§6.2.1) already removes a large fraction of the earlier serial host overhead and materially improves end-to-end wall throughput. The new measurements now show a split outcome: for narrow $H = 32$ bodies, a fully on-array ingress/egress path is worthwhile and scales especially well at larger batches, while the wider $H = 64$ accuracy model still leaves substantial wall time on the host side and has not yet been retrained in a corresponding full-NPU form. In parallel, genuine kernel-side gains remain valuable: increasing stream depth up to the per-DMA descriptor limit, overlapping the D2H of one chunk with the H2D of the next, or revisiting the MMUL tiling so that more compute is packed into each kernel invocation. The SRAM-sensitive scaling question is different: wider hidden bodies will require revisiting the first-tile ingress layout because that tile reaches the 64 KB ceiling before the internal tiles do. The highest-leverage next modeling step is to test whether the full-NPU ingress/egress strategy that works well for $H = 32$ can be extended profitably to the more accurate $H = 64$, $L = 32$ regime.

A AMD XDNA 2 background

The main text intentionally keeps only the hardware facts that directly matter for the systems argument. For readers less familiar with AI Engine-class NPUs, Figure 4 gives a programmer's logical view of one compute tile. It is not a physical floorplan. What matters here is that each tile combines local SRAM, a VLIW+SIMD control core, a tile-local MMUL datapath, and explicit DMA / FIFO connectivity. That combination makes weight-stationary streamed mappings natural on this hardware: weights can stay local, activations arrive through explicit buffers, and data movement must be planned rather than delegated to a large shared cache hierarchy [4, 3].

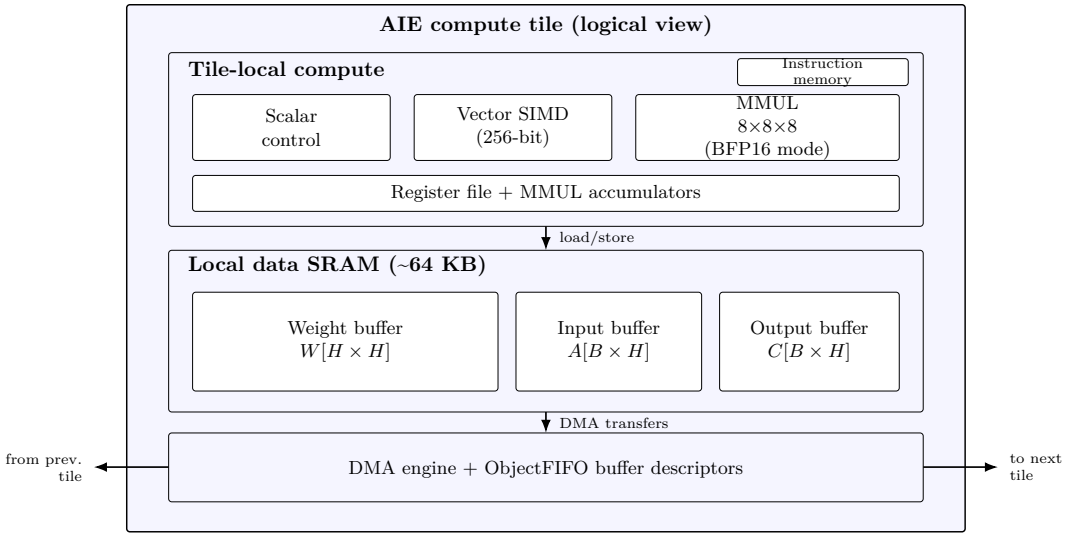

Figure 4: Logical view of one XDNA 2 compute tile from a programmer's point of view. The figure is schematic rather than a physical floorplan. What matters for this paper is the combination of tile-local SRAM, VLIW+SIMD control, MMUL-backed matrix math, and explicit DMA / FIFO movement between tiles.

Table 7 collects the hardware terms that recur most often in the paper. It is meant as a compact glossary, not an exhaustive one, but it gives enough background to follow the deployment discussion without already knowing the AI Engine vocabulary.

Term	Meaning in this paper
AIE tile	One compute tile of the XDNA 2 array: local SRAM, local compute, instruction memory, and DMA connectivity.
MemTile	Memory-only tile between shim tiles and compute rows; useful for routing and on-chip buffering.
Shim tile	Interface tile that connects the array to host-visible memory and runtime infrastructure.
DMA	Hardware transfer mechanism used to move data between host-visible buffers and tile-local SRAM without CPU-managed copy loops.
SIMD	Single Instruction, Multiple Data. One instruction operates on a vector of values at once.
VLIW	Very Long Instruction Word. One instruction can encode multiple parallel operations such as loads, arithmetic, and control.
MMUL	Tile-local matrix-multiply datapath used by the AIE kernel intrinsics; it strongly favors blocked 8×8 layouts.
ObjectFIFO	IRON abstraction for explicit producer/consumer channels between tiles and host boundary stages.
XCLBIN / XRT	The compiled device artifact and the runtime that loads and launches it on the NPU.

Table 7: Compact glossary for the AMD XDNA 2 terms that are used repeatedly in the main text.

B HIGGS physics background

The HIGGS benchmark comes from a standard collider-analysis setting: a rare target process must be separated from much more common look-alike processes using reconstructed event kinematics [6]. The label is binary. In the UCI release, '1' denotes signal and '0' denotes background [7]. Physically, that does not mean the detector has directly "seen a Higgs." It means the reconstructed pattern of visible objects in the event is more consistent with the target signal process than with ordinary background production.

Each row is therefore already one reconstructed collision event, not raw detector data. The low-level variables describe the visible ingredients of the event: one charged lepton, missing transverse momentum, and up to four jets, plus per-jet b-tag indicators [7]. In collider language, p_T is momentum transverse to the beam, η and ϕ encode direction, and missing transverse momentum records the momentum imbalance associated with invisible particles leaving the detector. The high-level variables are invariant-mass combinations such as m_{jj}, m_{jjj}, m_{lv}, m_{jlv}, m_{bb}, m_{wbb}, and m_{wwbb}; they test whether groups of reconstructed objects line up with a plausible underlying resonance or decay chain [7].

That structure is exactly why HIGGS is useful here. The model does not have to solve raw reconstruction. It receives a compact physics summary of each event and must make a fast signal-versus-background decision on dense numeric inputs. In this repository those examples come from the public `jxie/higgs` mirror of the published train/test split [8].

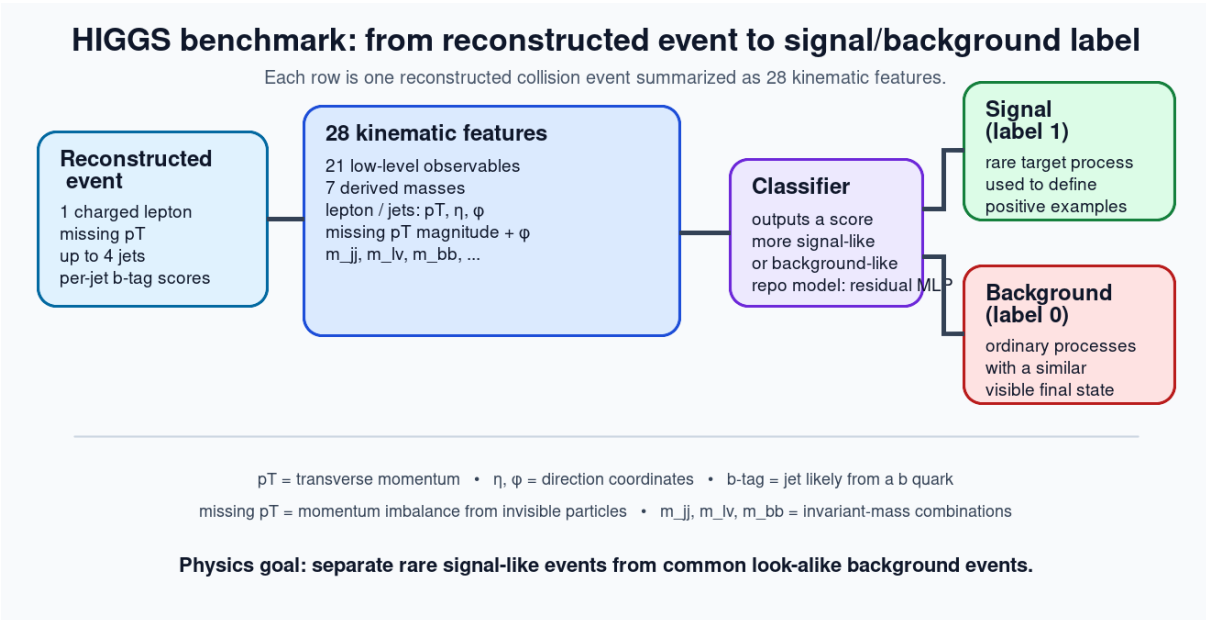

Figure 5: From reconstructed collision event to binary label in HIGGS. The dataset stores lepton, jet, missing-momentum, and derived invariant-mass features; the classifier must separate rare signal-like events from common look-alike background events.

Disclosure of AI-assisted workflow

AI coding and research assistants, including Claude Opus 4.6, GPT-5.4, and Gemini 3.1 Pro, were used extensively during this project to generate code, propose experiments, summarize results, review literature, and critique or draft text. The author curated the research direction, selected the final claims and figures, verified the reported results, and takes responsibility for the content of this paper.

References

[1] AMD, *IRON*.
https://github.com/amd/IRON

[2] Xilinx / AMD, *MLIR-AIE programming guide*.
https://github.com/Xilinx/mlir-aie/tree/main/programming_guide

[3] Linux kernel documentation, *AMD NPU / amdxdna*.
https://docs.kernel.org/next/accel/amdxdna/amdnpu.html

[4] AMD, *AI Engine documentation (programming model and architecture)*.
https://docs.amd.com/r/en-US/ug1273-versal-acap-design/AI-Engine

[5] *AMD XDNA overview*.
https://en.wikipedia.org/wiki/AMD_XDNA

[6] P. Baldi, P. Sadowski, and D. Whiteson, *Searching for Exotic Particles in High-Energy Physics with Deep Learning*.
https://arxiv.org/abs/1402.4735

[7] UCI Machine Learning Repository, *HIGGS dataset*.
https://archive.ics.uci.edu/dataset/280/higgs

[8] J. Xie, *HIGGS* (Hugging Face dataset mirror with published benchmark splits).
https://huggingface.co/datasets/jxie/higgs

[9] R. Ardino et al., *Development and demonstration of the CMS Phase-2 Level-1 trigger Data Scouting system*.
https://doi.org/10.1088/1748-0221/21/01/C01024

[10] J. Duarte et al., *Fast inference of deep neural networks in FPGAs for particle physics*.
https://doi.org/10.1088/1748-0221/13/07/P07027

[11] Y. Chen et al., *DaDianNao: A Machine-Learning Supercomputer*.
https://doi.org/10.1109/MICRO.2014.58

[12] Y.-H. Chen, J. Emer, and V. Sze, *Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks*.
https://doi.org/10.1109/ISCA.2016.40

[13] T. Chen and C. Guestrin, *XGBoost: A Scalable Tree Boosting System*.
https://arxiv.org/abs/1603.02754

[14] G. Ke et al., *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*.
papers.nips.cc/paper/6907-lightgbm

[15] L. Prokhorenkova et al., *CatBoost: Unbiased Boosting with Categorical Features*.
https://arxiv.org/abs/1706.09516

[16] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, *Revisiting Deep Learning Models for Tabular Data*.
https://arxiv.org/abs/2106.11959

[17] L. Grinsztajn, E. Oyallon, and G. Varoquaux, *Why do tree-based models still outperform deep learning on tabular data?*.
https://arxiv.org/abs/2207.08815

[18] K. He, X. Zhang, S. Ren, and J. Sun, *Deep Residual Learning for Image Recognition*, CVPR 2016.
https://arxiv.org/abs/1512.03385

[19] J. Duarte et al., *hls4ml: An Open-Source Codesign Workflow to Empower Scientific Low-Power Machine Learning Devices*, Front. Big Data 4, 2021.
https://doi.org/10.3389/fdata.2021.604443

[20] N. P. Jouppi et al., *In-Datacenter Performance Analysis of a Tensor Processing Unit*, ISCA 2017.
https://arxiv.org/abs/1704.04760

[21] S. Ö. Arik and T. Pfister, *TabNet: Attentive Interpretable Tabular Learning*, AAAI 2021.
https://arxiv.org/abs/1908.07442

[22] S. Popov, S. Morozov, and A. Babenko, *Neural Oblivious Decision Ensembles for Deep Learning on Tabular Data*, ICLR 2020.
https://arxiv.org/abs/1909.06312

[23] M. Feickert and B. Nachman, *A Living Review of Machine Learning for Particle Physics*, 2021.
https://arxiv.org/abs/2102.02770